Project - Deliverable 1

Ke Xin (Christina) Gao

Sheridan College

SYST17796 - Fundamentals of Software Design and Development

Professor Ronak Sheth

Nov 04, 2026

Table of Contents

1. Project Background and Description —---------- 3
2. Project Scope —---------- 5
3. High Level Requirements —---------- 6
4. Implementation Plan —---------- 7
5. Design Considerations —---------- 8
6. Starter Code UML —---------- 9
7. Group Contract —---------- 10

Project Background and Description

The goal of this project is to create a fully functional and multiplayer version of the card game “Go Fish” using Java.

Rules of Game:

1. The game must consist of 2-5 players, and uses a single deck of cards with Jokers removed (52 cards).
2. Each player starts with 5 cards in their hand. The remaining cards are in the draw deck, face-down.
3. Each player rolls a dice, the player with the highest number goes first.
    - Player turns are sequenced clock-wise.
4. On player’s active turn:
    - They can ask ANY other player for a specific value of card (e.g. 8).
        - If the other player has this value in their hand, they must hand over all cards of that value to the active player.
        - If the other player does not have this value, they say, “Go fish”. The active player will then draw 1 card from the deck.
    - The active player can create “books” containing 4 cards of equal value from their hand.
        - Books must be removed from the player's hand and placed on the table face-up.
5. The game is over when any player has no more cards remaining in their hand.
6. The winner is the player who ended the game with the highest amount of “books”.

Starter Code Analysis:

The starting code includes four .java files:

- Card.java
- Game.java
- GroupOfCards.java
- Player.java

So far, the project is made up of abstract classes with basic card game structure but without specific behaviours and methods designed for a particular game. The data members are private and are accessed through accessors and mutators. In GroupOfCards.java, the cards list is not initialized in the constructor. This would cause a NullPointerException when calling certain methods such as shuffle() or getCards(). Overall, the starting code is quite incomplete in both game logic.

Project Scope

Group Members and Roles:

Ke Xin (Christina) Gao (student number: 991610458)

I am the sole member of this group, and I will complete the entire scope of the project.

Technical Scope:

This game will run from the output of a console. Player turns will be executed through text input, and the game state will also be displayed through text formatting. Since there will only be a singular console shared amongst all the players, an interface will be created to prompt for player turns and show only the necessary information to the active player. Additionally, during all player turns, they will be able to see the draw deck size, number of cards in other players' hands, the number of "books" attributed to each player, and their own hand.

The project will be considered completed after the following criteria has been met:

- A custom game size of 2, 3, 4, or 5 players can be created.
- The rules are understandable to the players involved.
- The rules outlined in Project Background are implemented and verified through tests (e.g. invalid inputs are not accepted).
- Classes within this project follow OOP principles and demonstrate loose coupling, high cohesion.
- UML diagrams depict the Java code correctly.
- All files are committed to the repository.
- The full game can be played from start to finish without errors and crashes.

High-Level Requirements

Go Fish new system requirements:

1. Ability for 2-5 players to register in a “game lobby” with unique player names.
2. Ability to create and shuffle a deck of 52 cards and deal an appropriate amount of cards to each player.
3. Ability for each player to roll a die, and reroll upon ties.
4. Ability for the active player to target any other player during their turn.
5. Ability for the active player to ask for a card of any value.
6. Ability for the system to check for card value in targeted player’s hand and transfer cards or signal “Go fish”.
7. Ability to detect 4 cards of the same value in a player’s hand and remove them upon player request.
8. Ability to record player-created “books” on the playing board.
9. Ability for players to draw from the draw deck.
10. Ability to advance turns clockwise after the active player declares “End turn”.
11. Ability for the active player to see all appropriate information regarding the game state and their own hand.
12. Ability to detect the end of the game, calculate the winner, and declare the winner.

Implementation Plan

Project Github:

https://github.com/christinakgao/syst17796-project

Comments are expected with each commit. Pushes to the remote repository are to be made at the end of a working day after reviews are made on any changes.

Notable directories in the project include:

- /src - for all Java course code
- /docs - PDFs, text, and word documents related to the project (such as Group Contract and Design Template)
- /uml - UML diagrams

Tools expected for this project:

- NetBeans
- Visual Paradigm

Coding standards to follow:

- OOP principles
- Loose coupling
- High cohesion
- Header comments that include:
    - Contributing member names
    - Title of project
    - Date of project
    - Summary of project
- Project comments that are concise and accurately provide information on methods/variables

Design Considerations

Encapsulation:

- The current starter code has private data members.
    - E.g. Player.java has data member ‘private String name;’ and method ‘public String getName()’.
- The starter code is currently structured with abstract classes.
- For this project, I will likely create additional classes such as ‘Hand.java’ in order to provide a controlled method to modify cards in players’ hands.
- Need to create validation for player names and player inputs (not null, fits the parameters of the game).

Delegation:

- Game.java delegates to Player.java. The Game.java class stores the players and allows each player to individually implement actions within the game.
- Currently the shuffling is delegated to GroupOfCards.
- Formatting will be delegated to each class’ toString(), however, console output will be a separate class on its own to ensure that game logic is not mixed up with game display.

Flexibility/Maintainability:

- Since the base classes Card.java, Game.java, and Player.java are abstract, they are reusable in other game formats. This game will extend these abstract classes rather than modify them.
- GroupOfCards.java also has flexibility in its size, which allows for it to be used in a multitude of card games that have decks of different sizes.
- Need to have data members like Suit and Value represented with enums and constants rather than lists of hard-coded Strings.

Starter Code UML Diagram

UML Diagram files are also uploaded onto the Git repository.





